DS5619 MACHINE LEARNING SYSTEMS OPERATIONS

COUNSEL

TEAM MEMBERS

M.VIVEK CHAITHANYA (142301024)

J.SAI VARUN (142301040)

K. MD YASIN (142301017)

K. ADARSH REDDY (102301018)

THE PROBLEM

A strong research agent, but not production-ready

IA smart elevator dispatcher. At 9 AM, 1,000 workers arrive: one small elevator means long queues (SLO breach), ten giant empty ones waste electricity (budget overrun). COUNSEL picks the right VM size for each service.

Firewall
filters HTTP traffic

Web server
UI and business logic

Database
executes transactions

150 ms end-to-end latency SLO
2,500–3,600 resource cost budget
6 VM flavors to choose from per step

Original system

Hegde et al., CCGrid 2023

GCN-PPO reads a 3×8 graph state and picks 1 of 6 VM flavors

Reproduced: 1,000 epochs on the Madhava GPU cluster (job #93812)

Baseline: return 555.35, 0 budget overruns

Strength: captures service dependencies and balances SLO vs budget

Limit: one PyTorch model hard-coded in inference.py

What was missing for production

1. **No modularity** locked to one model
2. **No input validation** raw arrays go straight in
3. **No stored data** simulator states vanish
4. **No serving layer** no API, cache or hot-swap
5. **No observability** no signals, no drift tracking

ARCHITECTURE

Six decoupled stages, each with its own contract

1

Config & contracts

`schema.py`

Quoted scalars, Pydantic checks

2

Feature store

`feature_store.py`

9,000 steps in Parquet

3

Model registry

`adapters/`

5 models, one interface

4

Evaluation & gate

`evaluate_swap.py`

45,000 steps scored

5

Serving API

`app.py`

LRU cache, live swap

6

Monitoring & drift

`monitor.py`

PSI alerts, retrain trigger

Orchestration and QA

DAG pipeline: tasks run in dependency order in 10.02 s

GitHub Actions: lint, 14 tests and a release gate on every push

Hot-swap: change the live model without downtime

Closed-loop retraining

Observe: four golden signals and PSI on live telemetry

Alert: PSI above 0.25 means critical drift

Retrain: rebuild the Parquet data and train again

DATA QUALITY

Bad data is stopped at the door

Example: Airport security for model inputs. Bad data is stopped at the gate before it ever reaches the model.

configs/config.yaml

One source of truth: SLO 150 ms, budget [2500, 3600]

The Norway problem: YAML 1.1 reads bare NO, off, on, yes as booleans, so country code NO becomes False

Fix: quote the scalars, and schema.py verifies it

```
country: NO      # parsed as False
country: 'NO'    # stays the string
```

schema.py

Pydantic contract: 3×8 state plus a 6-value action mask

Rejects NaN, Inf and negative arrival rates with DataContractError

Avro rule: every new field gets a default, for forward and backward compatibility

Action allowlist: never delete a VM that isn't running

Raw 3×8 state → `validate_observation()` → Model predicts → `validate_action()`

9,000 / 9,000 simulation steps validated · 0 violations

FEATURE STORE

One frozen dataset for every model

In plain English
A certified grocery pantry. You can't cook a good meal from unlabelled, spoiled ingredients, so we freeze 9,000 simulator steps into clean, timestamped files that every model shares.

9,000
validated rows

55
columns per row

20
episodes, seed 42

3
timestamps per row

How it's built

Design: one shared pipeline for training and serving

Source: 20 PPO episodes saved to offline_store.parquet

55 columns: state, action mask, action, reward, SLO breach, resource use

Three clocks: event time, ingestion time, processing time

Point-in-time split



Cutoff 2026-11-06T10:00:00Z. Models never see data after it.

Lineage: metadata.json holds SHA-256 hashes of config, model and dataset, traced to the seed-42 PPO run

MODEL REGISTRY

Any policy plugs in through one function

```
predict(obs, mask) -> int
```

One interface for every policy. Switching models is a one-line change in config.yaml, with no change to simulator code.

1

GCN-PPO

Online RL baseline

13,383 params
~3.3 ms

Reads the service graph. Trained 1,000 epochs.

2

Heuristic

Rule-based

0 params
~0.3 ms

Scale up above 80% load, down below 45%.

3

Random Forest

Classical ML

120 trees
~9.7 ms

An action forest plus a risk forest that upgrades risky actions.

4

PatchTST

Time-series transformer

407,372 params
~6.0 ms

Reads the last 16 steps of 10-channel telemetry.

5

Decision Transformer

Offline-RL transformer

998,565 params
~7.3 ms

Return-conditioned, with a target return of 750.

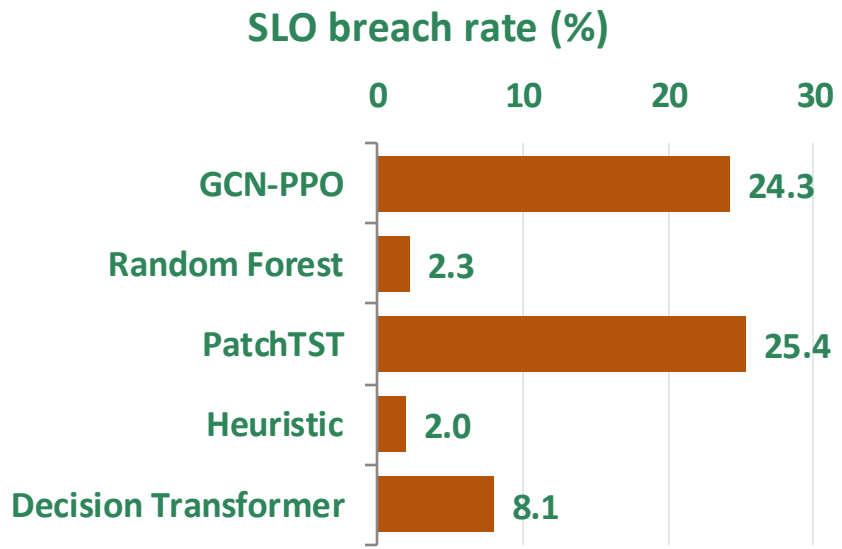
All offline models train on the same feature-store split.

```
93812        COUNSEL_J+        gpu   COMPLETED 2026-11-04T19:47:32 2026-11-04T23:04:52   03:17:20
```

EVALUATION

PPO scores highest; Random Forest is the safer trade

Model	Return	SLO breach	P99	Util.	Gate
GCN-PPO	555.35	24.27%	3.34 ms	58.6%	PASS
Random Forest	513.65	2.29%	9.69 ms	40.1%	PASS
PatchTST	477.34	25.36%	5.96 ms	53.1%	PASS
Heuristic	445.80	2.04%	0.34 ms	34.9%	PASS
Decision Transformer	444.57	8.11%	7.28 ms	38.7%	PASS



Online RL vs the offline store

PPO needed 4.96M simulator steps. Random Forest trains in seconds from Parquet and reaches 92.5% of PPO's return.

Safety vs utilization

PPO breaches the SLO 24.3% of the time. Random Forest cuts that to 2.29%, about 10× fewer; the heuristic reaches 2.04%.

All five models pass the release gate: return ≥ 350 and zero budget overruns.

SERVING

Zero-downtime serving with an LRU cache

Example: A chef's cheat sheet. If 500 customers order the same dish, don't cook it from scratch each time: look up the answer you already worked out.

Endpoints (app.py)

Method	Path	Purpose
POST	`/predict`	Validate state, return an action
POST	`/swap/{policy}`	Switch the live model, no restart
GET	`/health`	Active model, requests, latency
GET	`/metrics`	Cache hit ratio, throughput

< 0.25 ms
cache-hit latency

512
states in the cache

422
HTTP code for bad input

How a request is handled

1. **Validate** the 3×8 state and mask with schema.py
2. **Hash** policy + state with SHA-256 into a cache key
3. **Hit:** return the stored action instantly
4. **Miss:** run the model (about 0.3–10 ms), store the result

AUTOMATION · CI/CD

Three gates on every commit

Example: Three security checkpoints before takeoff. Code quality is checked first, then the interface contracts, and finally the live model performance.

1 Gate 1 · Lint

`flake8 (PEP 8)`

- Catches syntax errors, undefined names and bad imports
- Fails the build on any violation
- Keeps four contributors' code consistent

2 Gate 2 · Tests

`pytest · tests/test_all.py`

- Norway quoting; NaN, Inf and negative traffic rejected
- Action-mask allowlist
- Contract test for all 5 adapters
- LRU cache hit, live swap, PSI math, DAG sort

3 Gate 3 · Release

`Automated gate evaluator`

- Average return ≥ 350
- Zero budget overruns
- Applied to every model before it can ship

14 / 14 tests passed in 33.8 s · workflow: .github/workflows/ci.yml

MONITORING

Four golden signals and a drift alarm

Example: A smoke detector for traffic changes. If requests shift from daytime browsing to a flash sale, we are alerted before the model begins to fail.

Latency

P50 and P99 decision time per policy

Traffic

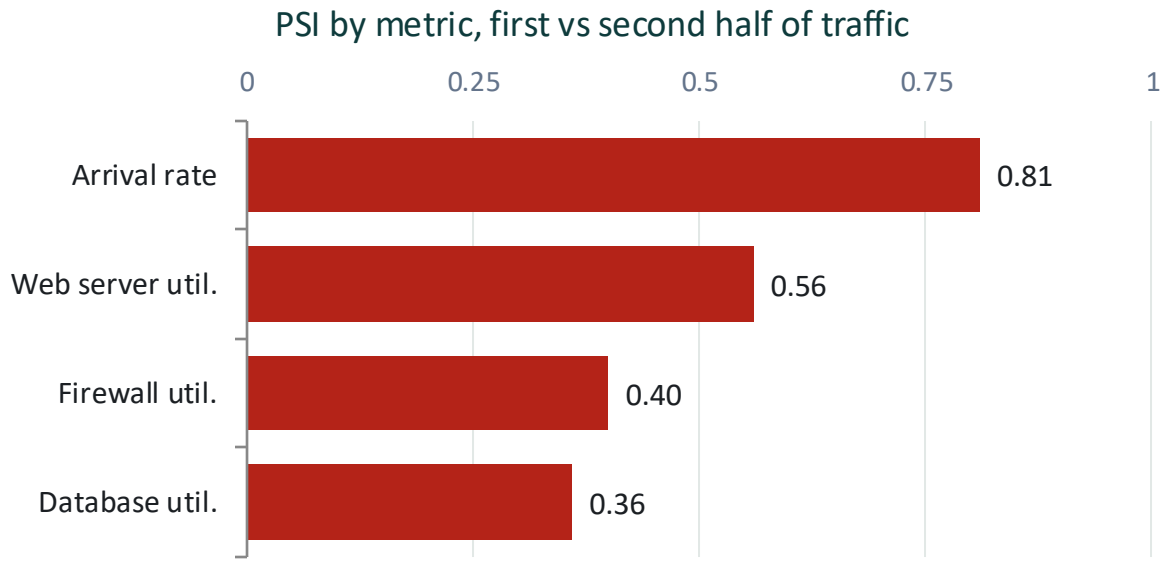
Steps and request arrival rate

Errors

SLO breaches against a 1% error budget, with burn rate

Saturation

CPU and memory per service; 0 budget overruns in 45,000 steps



PSI < 0.10 Normal

0.10 – 0.25 Warning

≥ 0.25 Critical: retrain

$$PSI = \Sigma\ (actual - expected) \times \ln(actual \div expected)$$

A simulated 2.8× traffic surge fires the critical alert, and our own data already shows drift between early and late episodes.

AUTOMATION · ORCHESTRATION

A dependency-ordered pipeline that runs in 10 seconds

Example: An automated assembly line. You can't paint a car before the chassis is built, so tasks run in a strict valid order and retry if a step glitches.

1	2	3	4	5	6
`validate_config`	`extract_features`	`score_policies`	`monitor_drift`	`model_card`	`notify_summary`
0.68 s	**6.75 s**	**0.001 s**	**2.59 s**	**0.00 s**	**0.01 s**
PASS	PASS	PASS	PASS	PASS	CERTIFIED

Kahn's topological sort

In-degree: a task runs only when its dependencies are done

Cycles: rejected before anything runs

Order: validate → extract → score policies → monitor, generate model card → summary

Resilience and telemetry

Retry with backoff: delay = base × factor^attempt

Transient faults: simulator socket disconnects are retried

Telemetry: each run saved to logs/pipeline_execution.json

Total 10.02 s · status RELEASE CERTIFIED

Three commands verify the whole system

1

```
python pipeline_dag.py
```

Run the full pipeline

Kahn's sort resolves dependencies, runs contracts, scores all 5 models and checks drift. Expect 6/6 tasks in about 10 s and RELEASE_CERTIFIED.

2

```
pytest tests/test_all.py -v
```

Run the quality gates

14 tests cover Norway safety, NaN/Inf rejection, masks, 5 adapters, LRU cache and the release gate. Expect 14 passed in about 34 s.

3

```
python monitor.py
```

Open the SRE dashboard

Computes the golden signals, error-budget burn and PSI, then writes charts/psi_drift_report.png.

WRAP-UP

What we delivered

5
swappable models

9,000
validated rows

14 / 14
tests passing

0
budget overruns

Capstone deliverables

1. **Baseline reproduced:** 1,000 epochs on the Madhava cluster, return 555.35
2. **Extension:** plug-in policy adapters across 5 model families
3. **Monitoring:** golden signals, error budget and PSI over steps
4. **Compute budget:** GPU job #93812 vs a 10 s pipeline run

Takeaways

PPO has the best return (555.4), but Random Forest keeps 92% of it with 2% SLO breaches

Next steps that can be done: online learning from live traffic and automatic retraining on drift

TEAM CONTRIBUTION

1 VARUN

Data & contracts

- config.yaml with quoted scalars
- schema.py and action masks
- Parquet feature store, 9,000 steps
- SHA-256 lineage, point-in-time split

2 ADARSH

Models & adapters

- adapters/base.py interface
- Five model wrappers, offline training
- evaluate_swap.py, 45,000 steps
- 6-section MODEL_CARD.md

3 YASIN

Serving & CI/CD

- app.py: predict, swap, health, metrics
- LRU cache and HTTP 422 validation
- Zero-downtime hot-swap
- ci.yml with 3 gates, 14 tests

4 VIVEK

SRE & pipeline

- monitor.py: four golden signals
- Error-budget burn, PSI dashboard
- pipeline_dag.py: Kahn's sort, backoff
- Telemetry log for each run

THANK YOU